TDD and BDD

Why testing?

Software development has transitioned from a waterfall to an Agile approach over the past decade. Since Agile development involves continuous change, testing has become invaluable to prevent these frequent changes from introducing new bugs, or worse, breaking the application in production.

Test Driven Developement

(TDD)

TDD is a software development technique that involves writing automated test cases prior to writing functional pieces of the code. This is popular in agile methodologies as it drives delivering a shippable product at the end of a sprint. The tests should be written before the functionality that is going to be tested.

Process Of TDD

Write a test case

Based on the requirements, write an automated test case.

Run all the test cases

Run these automated test cases on the currently developed code.

Develop the code for that test cases

If the test case fails, then, write the code to make that test-case work as expected.

Run test cases again

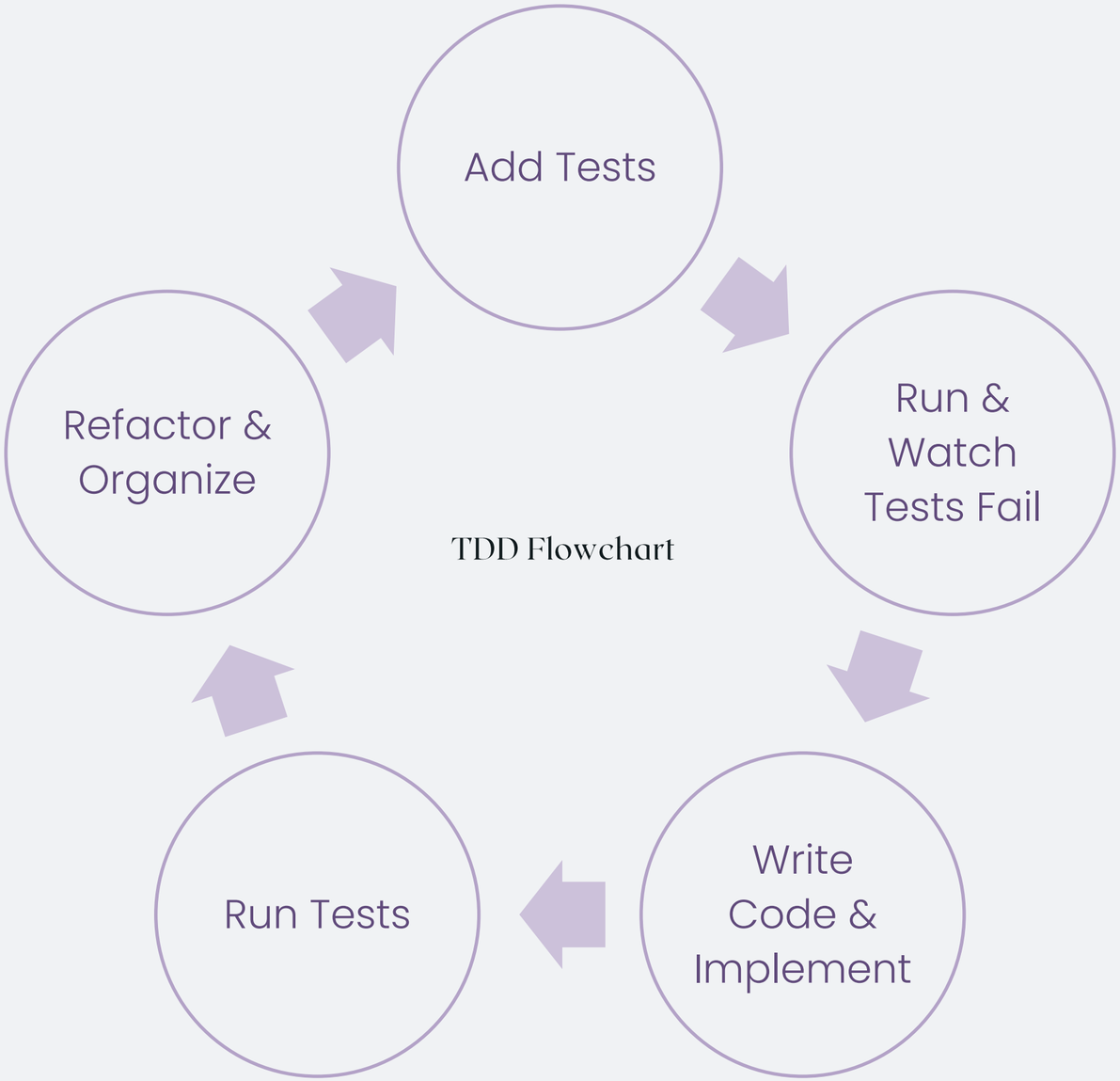
Run the test cases again and check if all the test cases developed so far are implemented.

Refactor your code

This is an optional step. However, it's important to refactor your code to make it more readable and reusable.

Repeat 1- 5 for new test cases

Repeat the cycle for the other test cases until all the test cases are implemented.


Add Tests
Run & Watch Tests Fail
Write Code & Implement
Run Tests
Refactor & Organize
TDD Flowchart

Advantages Of

TDD Approach

Better program design and higher code quality

- Programmers first have to define a goal they will achieve with the code piece
- Invites developers to use small classes and functions for particular requirements.
- It levels up the quality because all the possible mistakes and errors are already taken into account.
- Your code has a clear structure and fits the test requirements

Detailed project documentation

When writing tests for particular requirements, programmers immediately create a strict and detailed specification. It already includes all the likely users' actions.

Code flexibility and easier maintenance

- When writing code, developers strive to get fewer bugs, which consequently means that less time is required for fixing them.
- Being a part of TDD process, refactoring stands for optimization of existing code to make it more readable and easy to introduce. It encourages a pure and attractive code structure.

Advantages Of

TDD Approach

With TDD you will get a reliable solution

With TDD, we can be sure about the reliability of the developed solution. That is, tests help to understand if everything goes right after refactoring or adding a new feature.

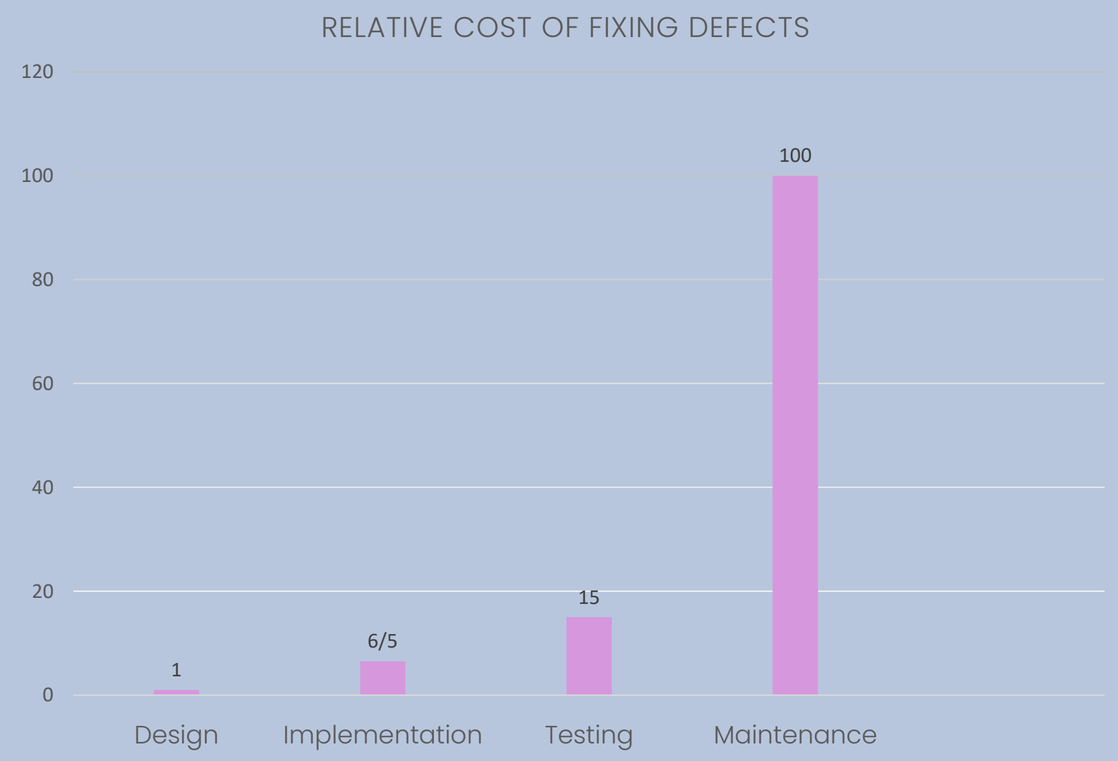
TDD reduces the time required for project development

- TDD approach allows identifying any issues very fast due to fast feedback.
- TDD approach offers is fewer bugs or errors.

Save project costs in the long run

And last but not least - with TDD implementation, the cost of a project development will significantly decrease.

“Defects found in testing were 15 times more costly than if they were found during the design phase and 2 times more than if found during implementation.”



Is TDD Enough?

Behavior Driven Developement

(BDD)

BDD stands for Behavior Driven Development. BDD is an extension to TDD where instead of writing the test cases, we start by writing a behavior. Later, we develop the code which is required for our application to perform the behavior.

The behavior of the application is the center of focus in BDD and it forces the developers and testers to walk-in the customer's shoes.

User Story & Processes

Process Of BDD

Write the behavior of the application

The behavior of an application is written in simple English like language by the product owner or the business analysts or QAs.

Write the automated scripts

This simple English like language is then converted into programming tests.

Implement the functional code

The functional code underlying the behavior is then implemented.

Check if the behavior is successful

Run the behavior and see if it is successful. If successful, move to the next behavior otherwise fix the errors in the functional code to achieve the application behavior.

Refactor or organize code

Refactor or organize your code to make it more readable and re-usable.

Repeat the steps 1-5 for new behavior

Repeat the steps to implement more behaviors in your application.

TDD vs. BDD

TDD	BDD
Stands for Test Driven Development.	Stands for Behavior Driven Development.
The process starts by writing a test case.	The process starts by writing a scenario as per the expected behavior.
TDD focuses on how the functionality is implemented.	BDD focuses on the behavior of an application for the end user.
Test cases are written in a programming language.	Scenarios are more readable when compared to TDD as they are written in simple English format.
Changes in how the application functions impact a lot on the test cases in TDD.	BDD scenarios are not much impacted by the functionality changes.
Collaboration is required only between the developers.	Collaboration is required between all the stakeholders.
Might be a better approach for projects which involve API and third-party tools.	Might be a better approach for projects which are driven by user actions. For eg: e-commerce website, application system, etc.
Some of the tools which support TDD are: JUnit, TestNG, NUnit, etc.	Some of the tools which support BDD are SpecFlow, Cucumber, MSpec, etc.
Tests in TDD can only be understood by people with programming knowledge,	Tests in BDD can be understood by any person including the ones without any programming knowledge.
TDD reduces the likelihood of having bugs in your tests.	Bugs in tests are difficult to track when compared to TDD.

Conclusion

Choosing between TDD Vs BDD can be very tricky. Some might argue that BDD is better for finding bugs whereas the others might just say that TDD gives higher code coverage.
Neither methodology is better than the other. It depends on the person and the project team to decide on which methodology to use. Also there is absolutely no harm in implementing both approaches – one to support the quality of the code the developer writes, and the other to support the behavior of the system defined by the product owner.

References

Dawson, Maurice & Burrell, Darrell & Rahim, Emad & Brewster, Stephen. (2010). Integrating Software Assurance into the Software Development Life Cycle (SDLC). Journal of Information Systems Technology and Planning. 3. 49-53.

D. Chekalin, “Advantages of Test Driven Development,” *Codica*, 23-Aug-2018. [Online]. Available: https://www.codica.com/blog/test-driven-development-benefits/. [Accessed: 12-Dec-2020].

J. Nair, “TDD vs BDD - What's the Difference Between TDD and BDD?,” *TestLodge Blog*, 11-Sep-2018. [Online]. Available: https://blog.testlodge.com/tdd-vs-bdd/. [Accessed: 12-Dec-2020].

P. J. Stevens, “BDD versus TDD - Understand the Difference: Cucumber Blog,” *BDD versus TDD - Understand the Difference | Cucumber Blog*, 07-Mar-2019. [Online]. Available: https://cucumber.io/blog/bdd/bdd-vs-tdd/. [Accessed: 12-Dec-2020].

“TDD Vs BDD - Analyze The Differences With Examples,” *Software Testing Help*, 13-Nov-2020. [Online]. Available: https://www.softwaretestinghelp.com/tdd-vs-bdd/. [Accessed: 12-Dec-2020].

T. Prakash, "Test Driven Development vs Behavior Driven Development | GlowTouch", *GlowTouch*, 2020. [Online]. Available: https://www.glowtouch.com/test-driven-development-vs-behavior-driven-development/. [Accessed: 12- Dec- 2020].